

Rekayasa Perangkat Lunak

Pertemuan 1: Evolusi dan Konsep Dasar Software Engineering

Memahami fondasi ilmu yang membangun dunia digital — dari filosofi, sejarah, hingga praktik profesional.



PT Nippon Indosari Corpindo Tbk



PT Indosari Niaga Nusantara

Adithya Marhaendra Kusuma

IT Support · Application Support · SCM · Data Control



 Sidoarjo

Candi, Sidoarjo

 Email

adit.marhaendra@gmail.com

 Telepon

082131009200

 LinkedIn

linkedin.com/in/adithya1994



PT Indosari Niaga Nusantara (Sari Roti Group)

SCM Head Control Tower (SPV) — Apr 2025–Sekarang

- Monitoring sinkronisasi data Distributor, Agen, dan Gudang DC
- Mengolah data harian: produktivitas, penjualan, inventaris, dan AR
- Menyusun laporan fulfillment delivery, OTA, mutasi stok, dan retur
- Memastikan kesesuaian data sistem dengan kondisi fisik barang

Regional Application Support (SPV) — Agt 2021–Apr 2025

- Operasional, konfigurasi, dan pemeliharaan sistem aplikasi perusahaan
- Troubleshooting aplikasi & audit data (data hygiene) master data
- Menyusun dokumentasi teknis, SOP, dan manual sistem
- Merancang dan memberikan pelatihan teknis kepada user baru

Pengalaman Mengajar

Universitas Yos Soedarso

2023–2026 Dosen Teknik Informatika — perencanaan, pelaksanaan, dan evaluasi pembelajaran.

UIN Sunan Ampel Surabaya

2022–2025 Dosen Tutor ICT — membuat materi, membimbing praktik, dan mengevaluasi hasil belajar.

Universitas Negeri Surabaya

2025 Dosen Tamu — memberikan materi kuliah industri.

Tujuan Pembelajaran

Setelah menyelesaikan materi ini, mahasiswa diharapkan mampu:

01

Mendefinisikan Filosofi RPL

02

Menganalisis Karakteristik Software vs Hardware

Membedakan produk rekayasa fisik dengan produk digital.

03

Mengidentifikasi Komponen Software Modern

Mengenali elemen penyusun perangkat lunak profesional.

04

Menjelaskan Software Crisis

Memahami latar belakang historis lahirnya disiplin RPL.

05

Memahami Spektrum Peran Tim

Mengenal peran dalam tim pengembangan perangkat lunak profesional.

BAB 1

Filosofi & Definisi RPL

Apa sebenarnya yang dimaksud dengan Rekayasa Perangkat Lunak?

5 Prinsip Filosofi RPL

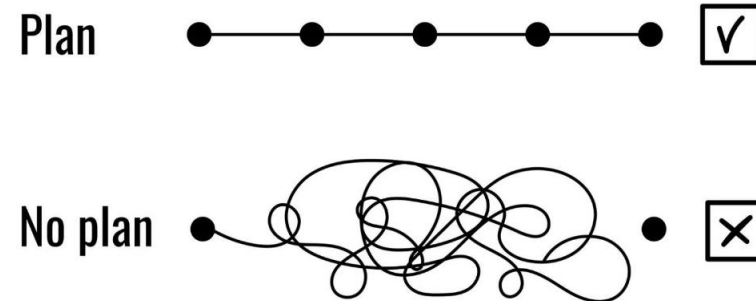
Berikut adalah lima prinsip utama yang menjadi landasan berpikir dalam Rekayasa Perangkat Lunak:

- 1 Dibangun Secara Sistematis**
Software tidak boleh dibuat secara asal-asalan. Setiap langkah harus mengikuti siklus hidup pengembangan (SDLC). *Coding tanpa perencanaan = potensi gagal tinggi.*
- 2 Berorientasi pada User**
Fokus utama adalah kebutuhan pengguna, bukan ego developer. Aplikasi secara teknis sempurna namun sulit digunakan tetap dianggap gagal.
- 3 Evolving System**
Kebutuhan selalu berubah seiring waktu. Software harus mudah di-maintain dan di-update. Software tidak pernah benar-benar "selesai."
- 4 Kualitas di Atas Segalanya**
Tidak cukup hanya bisa berjalan. Software harus aman, cepat, dan mudah digunakan — ketiganya harus terpenuhi secara bersamaan.
- 5 Kolaborasi adalah Kunci**
Software adalah hasil kerja tim: Developer, Tester, User, dan Project Manager bekerja bersama menuju tujuan yang sama.

Analogi: Membangun Rumah vs Membangun Software

🏠 Tanpa Blueprint

Bayangkan membangun rumah tanpa gambar desain — dinding tidak lurus, ukuran tidak presisi, dan hasilnya berantakan. Begitu pula software tanpa perencanaan: fitur tidak terintegrasi, bug menumpuk, dan proyek melebihi anggaran.



📐 Dengan Blueprint

Arsitektur rumah yang baik dimulai dari blueprint yang matang — setiap ruangan terencana, material terukur, dan proses pembangunan terarah. Software yang dibangun dengan filosofi RPL menghasilkan sistem yang kokoh, scalable, dan sesuai harapan pengguna.



💡 **Poin Penting:** Seperti arsitek yang tidak akan membangun gedung tanpa gambar teknik, seorang software engineer tidak seharusnya menulis satu baris kode pun tanpa perencanaan yang matang.

Definisi Rekayasa Perangkat Lunak

"A systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software."

— IEEE Standard Glossary of Software Engineering Terminology

Secara umum, **Rekayasa Perangkat Lunak (RPL)** adalah proses pengembangan perangkat lunak yang dilakukan secara sistematis, terstruktur, dan terukur untuk menghasilkan software berkualitas tinggi yang memenuhi kebutuhan pengguna dan pemangku kepentingan.

Systematic

Menggunakan langkah-langkah yang jelas, berurutan, dan dapat diulang dalam setiap proyek pengembangan

Disciplined

Mengikuti aturan, standar, dan metodologi yang telah ditetapkan oleh komunitas rekayasa perangkat lunak

Quantifiable

Setiap aspek dapat diukur secara objektif — waktu pengerjaan, biaya proyek, dan tingkat kualitas software

RPL Bukan Sekadar Menulis Kode

Rekayasa Perangkat Lunak adalah **disiplin ilmu teknik** yang menerapkan prinsip-prinsip engineering untuk menciptakan solusi digital secara sistematis, bukan hanya aktivitas *coding* semata.



Tiga Poin Utama RPL



Proses

Kerangka kerja yang menjaga **kualitas** dan **garis waktu** pengembangan tetap terkendali.



Metode

"Cara" teknis membangun software: analisis kebutuhan, desain arsitektur, dan pengujian.



Tools

Alat bantu otomatis atau semi-otomatis yang mendukung proses dan metode secara efisien.

BAB 2

Anatomi Perangkat Lunak

Software lebih dari sekadar file .exe atau aplikasi di ponsel Anda.

Tiga Komponen Kritisal Software



Dalam konteks profesional, software terdiri dari:

Instruksi (Program)

Kode sumber yang memberikan fungsi dan logika ketika dijalankan oleh mesin.

Struktur Data

Memungkinkan program memanipulasi dan menyimpan informasi secara efisien.

Dokumentasi

Menjelaskan operasi dan penggunaan sistem —
Technical Manual & User Manual.

Software vs Hardware: Perbedaan Fundamental

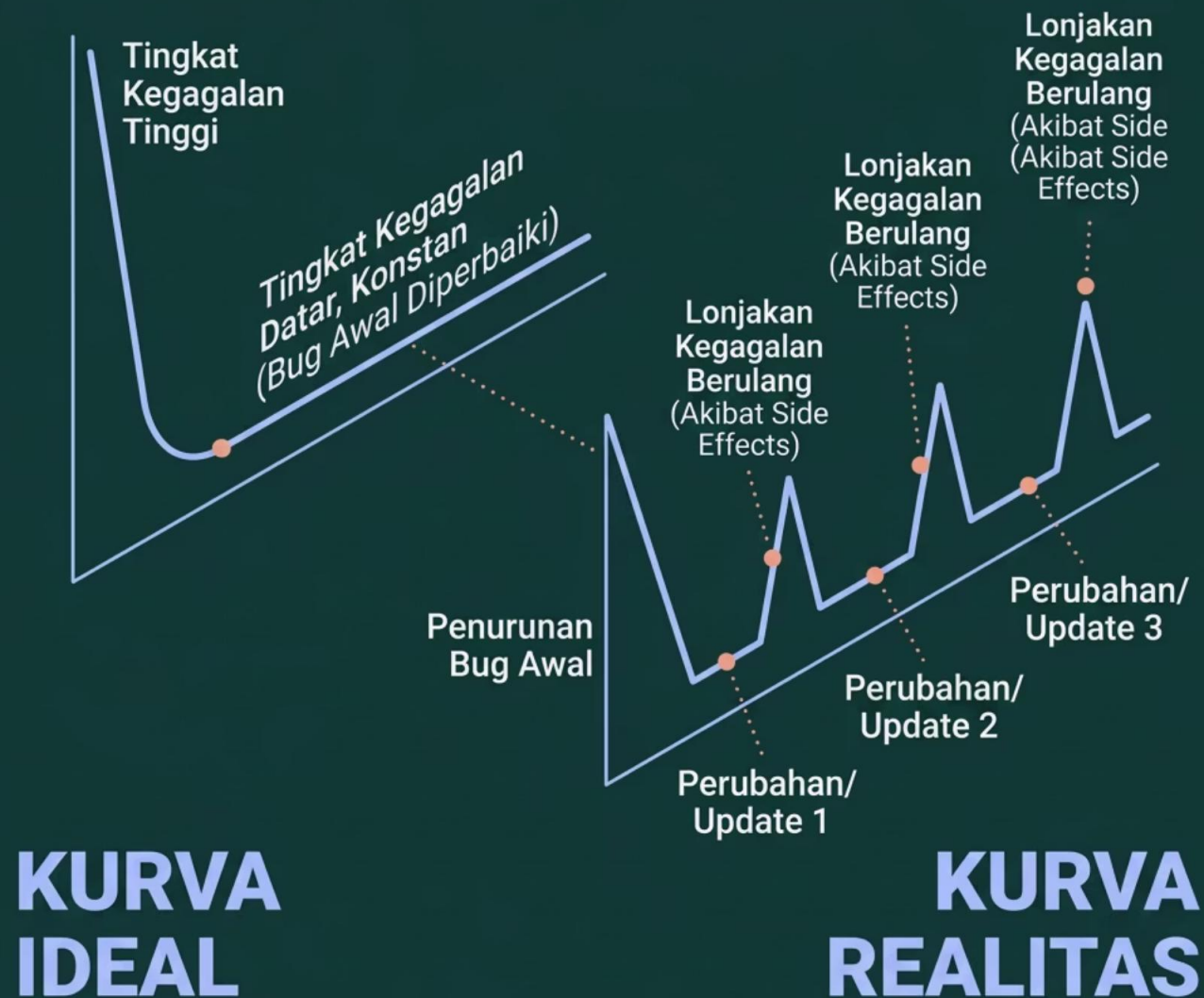
Dimensi	Perangkat Lunak (Software)	Perangkat Keras (Hardware)
Produksi	Dikembangkan / Direkayasa	Dimanufaktur di Pabrik
Kerusakan	Tidak "Aus", tapi "Memburuk" akibat perubahan	Bisa Aus karena gesekan fisik & panas
Kegagalan	Kesalahan desain / logika (Bug)	Kegagalan komponen fisik
Fleksibilitas	Sangat Tinggi — mudah diubah kodenya	Rendah — butuh desain ulang sirkuit

BAB 3

Kurva Kegagalan Software

Mengapa software semakin kompleks seiring berjalannya waktu?

Ideal vs Realitas: Kurva Kegagalan Software



Mengapa Berbeda?

Idealnya: Setelah bug awal diperbaiki, tingkat kegagalan menjadi datar dan konstan sepanjang waktu.

Realitasnya: Setiap perubahan atau *update* pada kode memicu lonjakan kegagalan baru akibat **efek samping (side effects)** yang tidak terduga.

- ❏ Berbeda dengan hardware yang mengikuti kurva "bak mandi" (pasti rusak setelah jam tertentu), software tidak aus — tetapi bisa memburuk akibat perubahan yang tidak terkelola.

BAB 4

Evolusi Jenis Software Modern

Spektrum perangkat lunak yang menggerakkan dunia saat ini.

7 Kategori Perangkat Lunak Modern



System Software

Melayani program lain: Compiler, OS, Driver



Application Software

Kebutuhan bisnis spesifik: POS, Payroll



Engineering Software

Algoritma berat: Simulasi orbit, Biologi molekuler



Embedded Software

Kontrol fungsi terbatas: Mesin cuci, Rem ABS

Kategori Software Modern (Lanjutan)



Product-Line Software

Software massal: Office suite, Grafik editor



Web / Mobile Apps

Berbasis browser dan perangkat genggam



AI Software

Algoritma non-numerik: Machine learning, Robotics

BAB 5

Krisis Perangkat Lunak

Latar belakang historis yang melahirkan disiplin RPL.

The Software Crisis (Akhir 1960-an)

Sebuah momen bersejarah yang memaksa dunia untuk **memikirkan ulang cara membangun software** secara serius.



Gejala-Gejala Software Crisis

Over Budget

Proyek perangkat lunak secara konsisten melampaui anggaran yang telah ditetapkan.

Over Time

Jadwal pengiriman selalu meleset, terkadang berbulan-bulan atau bahkan bertahun-tahun.

Kualitas Rendah

Software yang dihasilkan tidak efisien, penuh bug, dan sulit dipelihara.

Tidak Memenuhi Kebutuhan

Produk akhir sering tidak sesuai dengan apa yang sebenarnya dibutuhkan pengguna.

Mengapa Software Crisis Terjadi?



Ketimpangan inilah yang memaksa lahirnya RPL sebagai disiplin ilmu tersendiri untuk menjembatani kesenjangan tersebut.

Akar Masalahnya

Kompleksitas software **meningkat jauh lebih cepat** daripada kemampuan manusia untuk mengelolanya — tanpa adanya metode, proses, dan tools yang sistematis.



Inilah alasan utama RPL

diciptakan: untuk memberikan kerangka kerja ilmiah dan sistematis dalam membangun software yang andal.

BAB 6

Mitos Perangkat Lunak

Persepsi keliru yang sering merusak proyek software.



Software Myths: Persepsi Berbahaya

Mitos-mitos ini seringkali dipegang oleh manajemen atau pelanggan, dan jika tidak diluruskan, dapat **menghancurkan proyek** bahkan sebelum selesai.

Mitos 1: "Tambah Programmer, Proyek Beres"

Mitos

"Kalau kita terlambat, tambah saja programmer lagi dan proyek akan selesai lebih cepat."

Fakta: Brooks' Law

Menambah orang pada proyek yang **sudah terlambat** justru membuatnya **semakin terlambat**.

Alasannya: Setiap anggota baru membutuhkan waktu onboarding dan menambah **beban komunikasi** yang eksponensial bagi seluruh tim.

Mitos 2: "Software Jadi, Selesai Sudah"

Mitos

"Software yang sudah jadi dan dirilis bisa langsung ditinggal.
Pekerjaan sudah selesai."

Fakta: Biaya Maintenance

60% hingga 80% dari total biaya software justru terjadi **setelah** produk dirilis — pada tahap pemeliharaan (*maintenance*).

Bug, perubahan kebutuhan pengguna, dan pembaruan teknologi terus memerlukan perhatian tim pengembang.

BAB 7

Studi Kasus Kegagalan Software di Indonesia

Belajar dari kesalahan nyata untuk membangun sistem yang lebih baik.

Kasus 1: Serangan Ransomware PDNS 2

Juni 2024 — Pusat Data Nasional Sementara, Surabaya



Apa itu PDNS 2?



Latar Belakang Sistem

Pusat Data Nasional Sementara (PDNS) 2 berlokasi di Surabaya, dikelola oleh **Kemenkominfo** untuk mendukung integrasi data berbagai instansi pemerintah di seluruh Indonesia.

Pada Juni 2024, sistem ini menjadi target serangan **Ransomware Brain Cipher** — varian dari LockBit 3.0.

Dampak Serangan PDNS 2

Data Terenkripsi

Data milik **282 instansi pemerintah** terkunci. Banyak data tidak memiliki backup di lokasi berbeda.

Layanan Imigrasi Lumpuh

Sistem imigrasi di seluruh bandara Indonesia mati total, memicu antrian panjang secara manual.

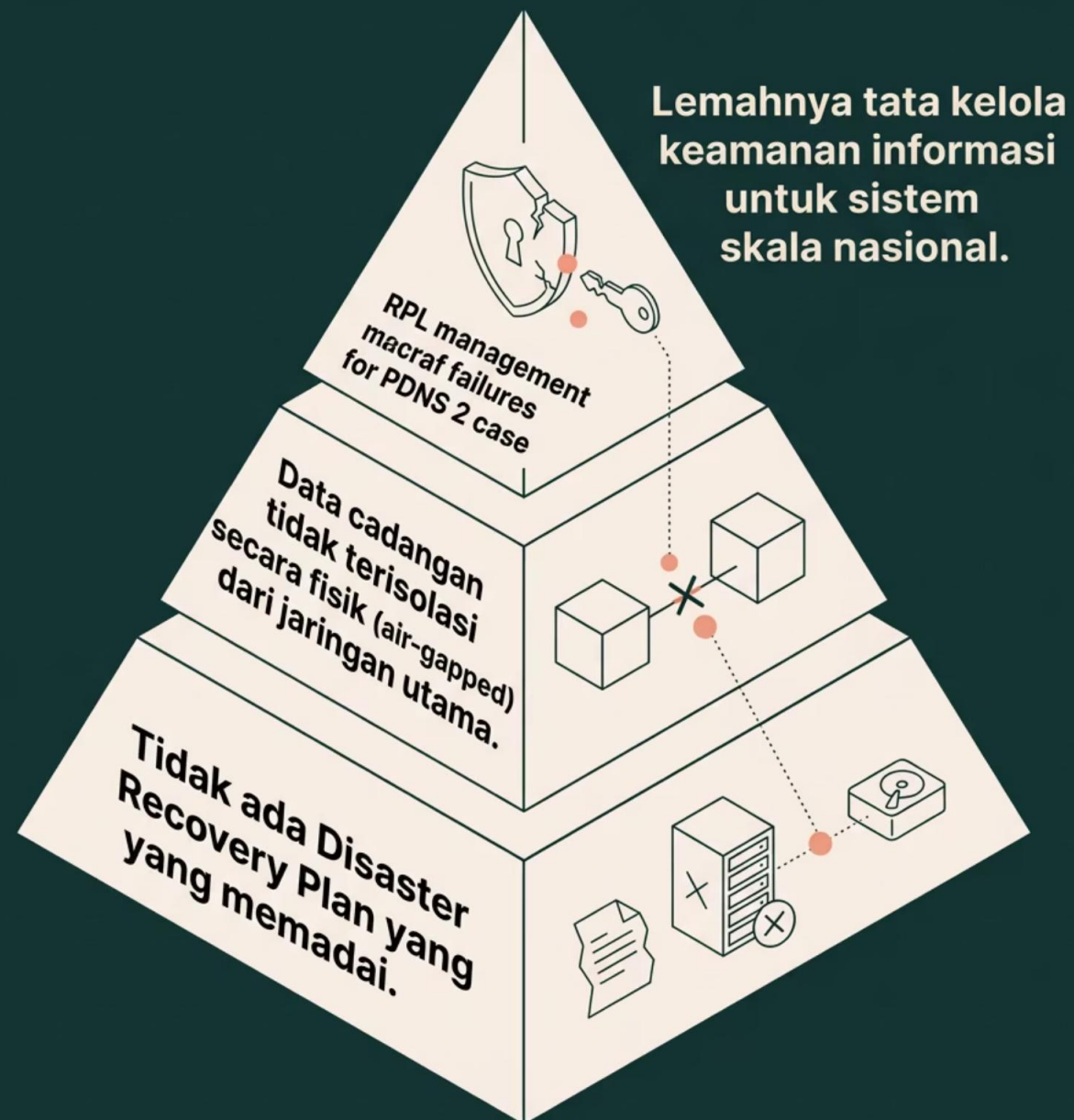
Gangguan Layanan Publik

Pendaftaran beasiswa KIP Kuliah dan sistem perizinan daerah ikut terdampak.

Kerugian Masif

Tebusan \$8 juta ditolak, namun biaya pemulihan dan kerugian produktivitas nasional diperkirakan sangat besar.

Analisis RPL: Kesalahan Manajemen PDNS 2



Akar Masalah: Kegagalan DRP

Secara teknis ada serangan siber, namun secara manajemen, sistem gagal menjalankan prinsip **redundancy** (cadangan data).

Fakta bahwa data cadangan tidak terisolasi secara fisik (**air-gapped**) dari jaringan utama menunjukkan lemahnya tata kelola keamanan yang seharusnya menjadi standar rekayasa sistem skala nasional.

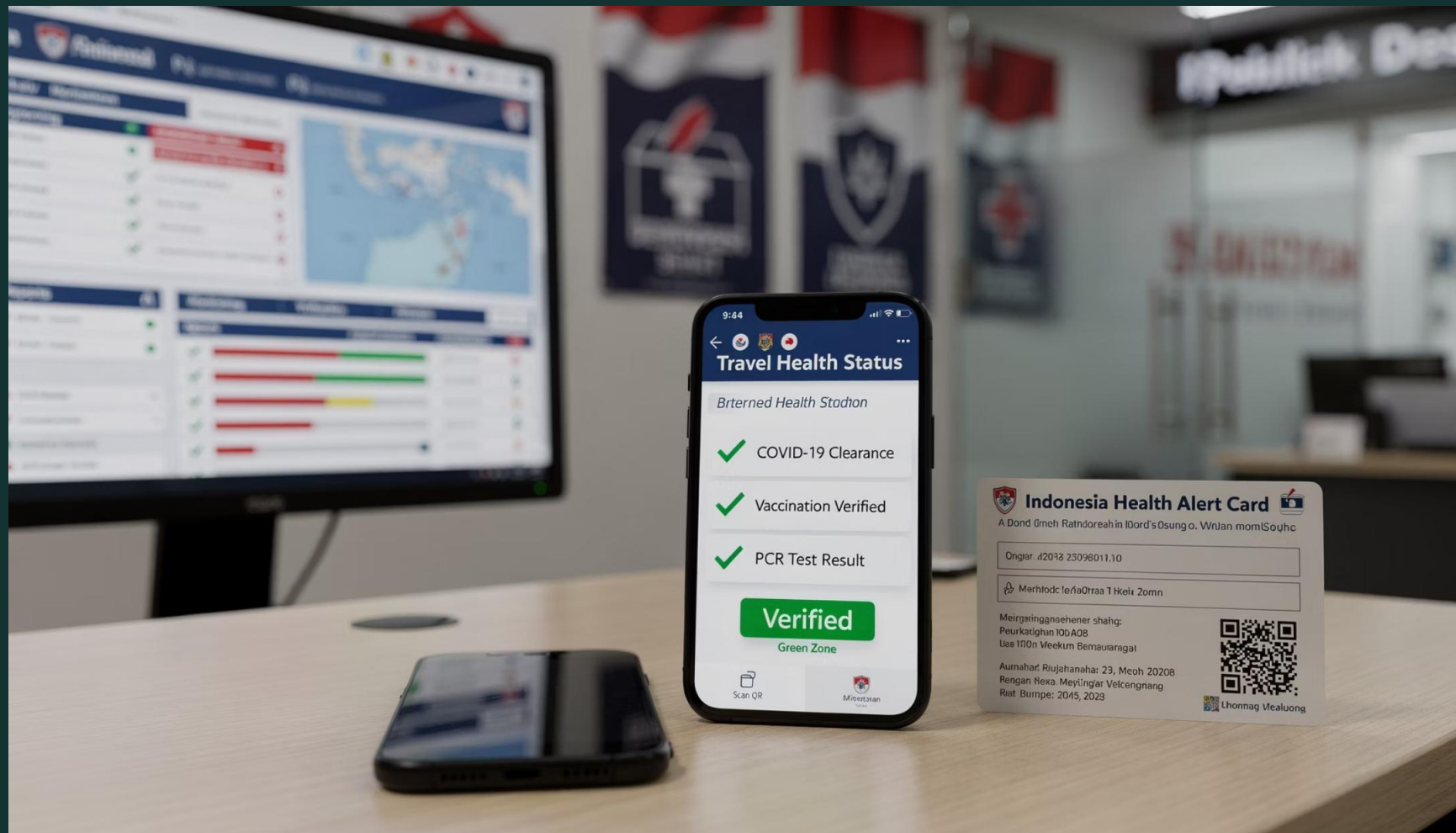
- ❑ **Pelajaran RPL:** Disaster Recovery Plan bukan opsional — ini adalah komponen wajib dalam manajemen sistem kritis.

Kasus 2: Kebocoran Data Aplikasi e-HAC

Agustus 2021 — Electronic Health Alert Card



Apa itu e-HAC?



Latar Belakang Sistem

e-HAC adalah aplikasi wajib bagi pelaku perjalanan selama pandemi COVID-19 untuk memantau **status kesehatan dan riwayat perjalanan**.

Pada Agustus 2021, ditemukan bahwa data pribadi jutaan pengguna **terbuka di database tanpa proteksi** — dapat diakses siapa pun yang mengetahui alamat servernya.

Dampak Kebocoran Data e-HAC

1.3 Jt

Data Pengguna Bocor

Data pribadi 1,3 juta pengguna terekspos ke publik tanpa enkripsi.

0

Proteksi Database

Database Elasticsearch dibiarkan terbuka ke publik — tanpa kata sandi maupun enkripsi.

Data yang bocor mencakup: **nama lengkap, NIK, nomor telepon, alamat, hasil tes COVID-19, hingga detail paspor** — data yang sangat sensitif.

Analisis RPL: Kesalahan Teknis & Manajemen e-HAC

Kesalahan Teknis

Konfigurasi Database Salah

Elasticsearch dibiarkan terbuka ke publik tanpa enkripsi atau otentikasi sama sekali.

Tidak Ada Security Testing

Tidak dilakukan *Penetration Testing* sebelum data pengguna masuk ke sistem produksi.

Kesalahan Manajemen

Lemahnya Pengawasan Standar

Tidak ada kepatuhan standar keamanan data yang diverifikasi sebelum aplikasi dirilis massal.

Prinsip RPL Dilanggar

Tahap Testing dalam siklus RPL seharusnya mencakup uji keamanan server sebelum data pengguna dientri.

Perbandingan Dua Kasus Kegagalan

Aspek	PDNS 2 (2024)	e-HAC (2021)
Jenis Kegagalan	Manajemen (DRP & Redundancy)	Teknis + Manajemen (Konfigurasi & Testing)
Dampak Utama	282 instansi lumpuh, layanan publik terhenti	1,3 juta data pribadi terekspos publik
Prinsip RPL Dilanggar	Disaster Recovery Plan, Redundancy	Security Testing, Penetration Testing
Pelajaran Utama	Backup data harus terisolasi (air-gapped)	Security testing wajib sebelum rilis

Pelajaran Bersama dari Kedua Kasus

Kedua kasus ini membuktikan bahwa **kegagalan software bukan hanya masalah teknis** — melainkan kegagalan dalam menerapkan prinsip-prinsip RPL secara menyeluruh sejak awal.





Bedah TikTok: Kacamata Rekayasa Perangkat Lunak

Di balik setiap video pendek yang menghibur, tersembunyi struktur rekayasa perangkat lunak yang masif dan kompleks. Mari kita bedah TikTok menggunakan prinsip-prinsip RPL untuk memahami bagaimana aplikasi dengan miliaran pengguna ini benar-benar bekerja.

Dekonstruksi 3 Elemen Perangkat Lunak TikTok

Dalam RPL, TikTok bukan sekadar "ikon di layar HP". Ia adalah kesatuan dari tiga elemen fundamental yang saling bergantung dan membentuk sistem yang utuh.



⚙️ Instruksi (Program / Fitur)

- **Algoritma FYP:** Machine Learning yang menentukan video berdasarkan perilaku pengguna
- **Video Editor:** Pemrosesan citra & suara secara real-time — filter wajah, sinkronisasi musik
- **CDN (Content Delivery Network):** Streaming video dari server terdekat agar bebas buffering



📁 Data (Informasi)

- **Profil Pengguna:** Nama, minat, lokasi, dan perangkat yang digunakan
- **Metadata Video:** Tagar (#), durasi, resolusi, dan musik
- **Behavioral Data:** Durasi tonton, kecepatan swipe, dan like — bahan bakar utama algoritma TikTok



📄 Dokumentasi

- **API Documentation:** Panduan bagi developer luar untuk integrasi login dan berbagi konten
- **System Architecture:** Dokumen komunikasi antar server TikTok global dengan pengguna di Indonesia



Karakteristik: Software "Memburuk" — Deterioration

TikTok terus berkembang — dari video 15 detik, kini mendukung durasi 10 menit, bahkan mengintegrasikan **TikTok Shop** ke dalam platform hiburan. Namun setiap perubahan membawa risiko nyata.

⚠ Risiko Perubahan

Ketika fitur Shop dimasukkan ke dalam aplikasi hiburan, struktur kode yang lama harus diubah secara menyeluruh. Arsitektur yang awalnya dirancang untuk streaming video kini harus menangani transaksi e-commerce secara bersamaan.

● Efek Samping (Side Effects)

Penambahan fitur belanja dapat membuat aplikasi lebih berat dan lambat, bahkan menyebabkan fitur kamera utama crash karena konflik memori. Inilah mengapa tim RPL TikTok wajib menjalankan **Regression Testing** setiap kali ada fitur baru dirilis.

📄 Regression Testing memastikan fitur lama tidak rusak akibat penambahan kode baru.

Simulasi Krisis Perangkat Lunak

Bayangkan manajemen TikTok meminta fitur "Live Streaming 4K" dirilis dalam 3 hari — tanpa analisis dan pengujian yang memadai. Apa yang terjadi?

🌟 Kegagalan Server

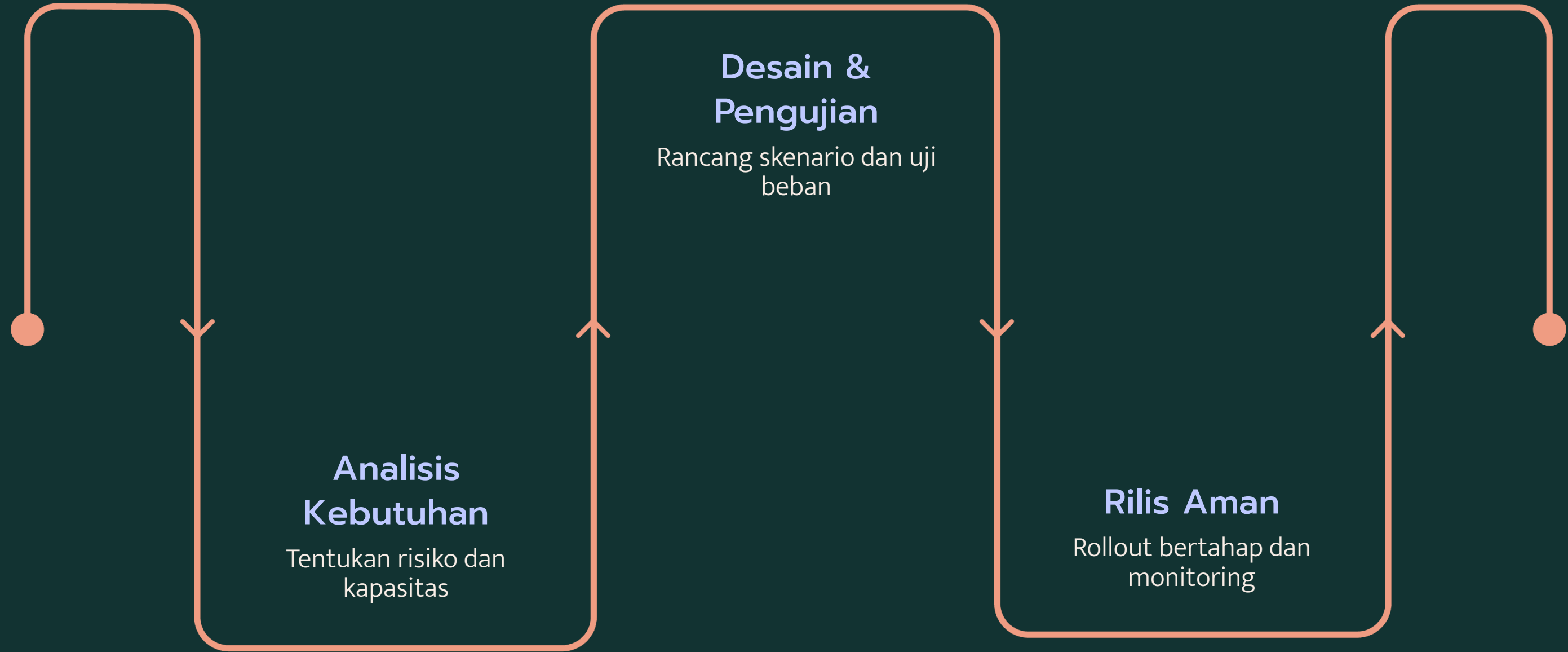
Server akan **down** karena tidak mampu menampung beban data 4K yang masif dari jutaan pengguna secara bersamaan. Infrastruktur tidak dirancang untuk lonjakan bandwidth sebesar itu.

📱 Kegagalan Pengguna

HP pengguna cepat panas dan baterai boros karena instruksi program tidak dioptimalkan. Kode yang tidak efisien memaksa prosesor bekerja jauh melebihi kapasitas normalnya.

✅ Solusi RPL

Seorang Software Engineer yang baik akan menegaskan: *"Kita butuh tahap Design dan Testing selama minimal 2 minggu untuk memastikan fitur ini tidak merusak sistem yang sudah berjalan."*



Pembagian Peran Tim: Roleplay Membangun Fitur TikTok

Jika kelompok mahasiswa membangun modul baru — misalnya fitur "**Komentar Suara**" di TikTok — begini pembagian peran dalam tim RPL yang ideal:



Analyst

Meneliti apakah pengguna TikTok lebih menyukai fitur "Komentar Suara" atau tetap memilih "Komentar Teks" — melalui survei, wawancara, dan analisis data perilaku pengguna.



Designer (UI/UX)

Merancang posisi tombol "Komentar Suara" agar tidak menghalangi tampilan video — mempertimbangkan estetika, aksesibilitas, dan kenyamanan pengguna.



Programmer

Menulis kode untuk menangkap input suara dari mikrofon pengguna, mengompres audio, lalu menyimpannya secara aman di database server TikTok.



Tester

Mencoba mengirim komentar suara selama 1 jam tanpa henti untuk mendeteksi apakah aplikasi mengalami force close, kebocoran memori, atau penurunan performa.



💡 Poin Diskusi Kelas: Mengapa RPL Itu Krusial?

"Jika TikTok tidak menggunakan prinsip RPL dan hanya asal coding — apa yang terjadi ketika 1 miliar orang membukanya secara bersamaan?"

Tanpa RPL: Bencana Digital

Aplikasi akan **hancur total** — server kewalahan, data pengguna korup, fitur-fitur bertabrakan satu sama lain. Inilah definisi nyata dari **Krisis Perangkat Lunak**.

Dengan RPL: Scalability Terjamin

RPL memastikan sistem mampu menangani lonjakan pengguna melalui arsitektur yang terencana, pengujian beban yang ketat, dan dokumentasi yang memungkinkan tim baru memahami sistem dengan cepat.



Scalability



Reliability



Performance

```
<html>
  <head>
    <meta charset="UTF-8" />
    <body {
```

THANK YOU

Slide Akhir: RPL Pertemuan 1

```
  }}
</body>
</body>
</html>
```